DevOps RAG Corpus

Corpus ID: `devops-rag-corpus`
Version: 1.0
Purpose: A consistent, text-based technical corpus for comparing Retrieval-Augmented Generation (RAG) frameworks.
Topics: Docker, Kubernetes, Terraform, AWS, and CI/CD.

> **Formatting note:** Code examples use fenced Markdown code blocks with language identifiers. Keep the opening and closing fences on their own lines when converting this file to PDF or another format.

---

1. Docker Fundamentals

1.1 What Docker does

Docker packages an application and its runtime dependencies into an image. A container is a running instance of an image. This helps teams reduce environment differences between development, testing, and production.

A Docker image is built from a Dockerfile. The Docker daemon or compatible builder reads the instructions, obtains base layers, executes build steps, and produces an image. Containers use the image as an immutable template while adding a writable container layer at runtime.

Common Docker concepts:

- **Image:** A versioned package containing application files and runtime dependencies.
- **Container:** An isolated process created from an image.
- **Registry:** A service for storing and distributing images.
- **Volume:** Persistent storage managed outside a container's writable layer.
- **Network:** A virtual network that allows containers to communicate.
- **Dockerfile:** A declarative set of instructions used to build an image.

1.2 A production-oriented Node.js Dockerfile

The following example uses a multi-stage build. Dependencies are installed in a build stage, while the final image contains only the files needed to run the application.

```
# Build stage
FROM node:22-alpine AS builder

WORKDIR /app

# Copy dependency manifests first to improve build-cache reuse.
COPY package*.json ./

# Install the exact dependency tree recorded in package-lock.json.
RUN npm ci

# Copy the application source and build it.
COPY . .
RUN npm run build

# Runtime stage
FROM node:22-alpine AS runtime

ENV NODE_ENV=production
WORKDIR /app

# Copy only production dependencies and compiled application output.
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY --from=builder /app/dist ./dist

USER node

EXPOSE 3000

CMD ["node", "dist/server.js"]
```

The `npm ci` command installs the dependency versions in the lock file and fails if the lock file and package manifest are inconsistent. The `--omit=dev` option excludes development dependencies from the runtime installation. A non-root user reduces the impact of a compromised application process.

The example assumes the project builds into `dist/` and starts from `dist/server.js`. Those paths must match the actual application.

1.3 Excluding unnecessary files

A `.dockerignore` file prevents local development files and secrets from entering the build context.

```
node_modules
npm-debug.log
.git
.gitignore
.env
.env.*
coverage
dist
Dockerfile
docker-compose.yml
README.md
```

Do not exclude build output if the Dockerfile expects to copy prebuilt output from the host. In the multi-stage example above, the image builds the output internally, so excluding local `dist/` is appropriate.

1.4 Build and run an image

```
# Build an image and assign a versioned tag.
docker build -t example-api:1.0.0 .

# Start a container and publish container port 3000 on host port 8080.
docker run -d \
  --name example-api \
  --publish 8080:3000 \
  example-api:1.0.0

# Inspect the running container.
docker ps

# Follow application logs.
docker logs --follow example-api

# Stop and remove the container.
docker stop example-api
docker rm example-api
```

The `--publish 8080:3000` option maps host port 8080 to container port 3000. A process listening only on `127.0.0.1` inside the container may not be reachable through the published port; applications commonly need to listen on `0.0.0.0`.

1.5 Docker Compose

Docker Compose defines a multi-container application in a YAML file. This example runs an API and a MySQL database.

```
services:
  api:
    build:
      context: .
    ports:
      - "8080:3000"
    environment:
      NODE_ENV: production
      DB_HOST: database
      DB_PORT: "3306"
      DB_NAME: appdb
      DB_USER: appuser
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      database:
        condition: service_healthy

  database:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: appdb
      MYSQL_USER: appuser
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 10

volumes:
  mysql_data:
```

Run the stack with:

```
docker compose up --build --detach
docker compose ps
docker compose logs --follow
docker compose down
```

`docker compose down` removes the containers and network but preserves named volumes by default. Add `--volumes` only when intentionally deleting persisted database data.

1.6 Operational and security practices

- Pin images to maintained versions and update them regularly.
- Prefer small, trusted base images, but do not choose an image solely because it is small.
- Run as a non-root user where possible.
- Keep credentials out of Dockerfiles and source control.
- Use health checks for meaningful service readiness.
- Set CPU and memory limits appropriate to the workload.
- Scan images and dependencies for known vulnerabilities.
- Send logs to a central logging platform.
- Rebuild images when base-image security updates are released.

---

2. Kubernetes Fundamentals

2.1 Kubernetes architecture

Kubernetes is a container orchestration platform. It manages workloads across a cluster and provides mechanisms for scheduling, service discovery, scaling, configuration, and recovery.

A Kubernetes cluster has a control plane and worker nodes.

- **API server:** The front end for Kubernetes API requests.
- **etcd:** A consistent key-value store for cluster state.
- **Scheduler:** Selects a suitable node for unscheduled Pods.
- **Controller manager:** Runs controllers that reconcile actual state with desired state.
- **Kubelet:** Ensures containers described by Pod specifications are running on a node.
- **Container runtime:** Runs containers, commonly through a CRI-compatible runtime.
- **kube-proxy or equivalent networking implementation:** Supports Service networking, depending on the cluster networking design.

Kubernetes is declarative. An operator describes the desired state in manifests, and controllers continuously work to make the observed state match it.

2.2 Deploy an application

A Deployment manages a ReplicaSet and maintains the requested number of Pod replicas.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-api
  labels:
    app: web-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-api
  template:
    metadata:
      labels:
        app: web-api
    spec:
      containers:
        - name: web-api
          image: nginx:1.27-alpine
          ports:
            - name: http
              containerPort: 80
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10
```

The selector must match the labels on the Pod template. Resource requests influence scheduling; limits constrain resource consumption. A readiness probe controls whether a Pod receives Service traffic. A liveness probe can trigger a container restart when the application is unhealthy.

Apply and inspect the Deployment:

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods -l app=web-api
kubectl describe deployment web-api
```

2.3 Expose the application with a Service

A Service provides a stable virtual endpoint for a changing set of Pods.

```
apiVersion: v1
kind: Service
metadata:
  name: web-api
spec:
  type: ClusterIP
  selector:
    app: web-api
  ports:
    - name: http
      port: 80
      targetPort: http
```

```
    protocol: TCP
```

The Service selects Pods using `app: web-api`. The Service port is 80, while `targetPort` refers to the named container port `http`.

For external HTTP access, a cluster may use an Ingress controller or a Gateway API implementation. Creating an Ingress resource alone does not install or configure an Ingress controller.

2.4 Configuration and secrets

ConfigMaps store non-confidential configuration. Secrets are intended for sensitive values, but base64 encoding is not encryption.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-api-config
data:
  LOG_LEVEL: "info"
  APP_MODE: "production"
```

A Pod can consume the ConfigMap as environment variables:

```
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
spec:
  containers:
    - name: app
      image: nginx:1.27-alpine
      envFrom:
        - configMapRef:
            name: web-api-config
```

For production secrets, enable encryption at rest where supported, apply least-privilege RBAC, restrict access to namespaces, and consider an external secrets manager.

2.5 Scaling and rolling updates

```
# Change the desired replica count.
kubectl scale deployment/web-api --replicas=5

# Inspect rollout status.
kubectl rollout status deployment/web-api

# View rollout history.
kubectl rollout history deployment/web-api

# Undo the most recent rollout if necessary.
kubectl rollout undo deployment/web-api
```

A rolling update gradually replaces old Pods with new ones. Readiness probes and appropriate rollout settings help prevent unavailable or unready Pods from receiving traffic.

2.6 Troubleshooting common workload problems

```
kubectl get pods -o wide
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl logs POD_NAME --previous
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get endpoints web-api
```

Typical symptoms include:

- `ImagePullBackOff`: the image name, tag, registry credentials, or network access may be incorrect.
- `CrashLoopBackOff`: the container repeatedly exits; inspect current and previous logs.
- `Pending`: the scheduler cannot place the Pod, often because of resource constraints, taints, or storage requirements.
- Service has no endpoints: check selectors, Pod labels, and readiness status.

---

3. Terraform and Infrastructure as Code

3.1 Infrastructure as Code

Infrastructure as Code (IaC) defines infrastructure in machine-readable configuration files. Terraform uses providers to interact with cloud platforms and other APIs.

A typical Terraform workflow is:

1. Write configuration describing the desired infrastructure.
2. Initialize providers and modules.
3. Validate configuration.
4. Generate and review an execution plan.
5. Apply the approved plan.
6. Inspect state and outputs.
7. Update or destroy resources through controlled workflows.

Terraform configuration is declarative. Terraform compares the desired configuration with its state and the real infrastructure to determine proposed changes.

3.2 A basic AWS provider and S3 bucket

The following example creates a private S3 bucket. The bucket name must be globally unique.

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  description = "AWS region for the infrastructure."
  type        = string
  default     = "eu-north-1"
}

variable "bucket_name" {
  description = "Globally unique S3 bucket name."
  type        = string
}

resource "aws_s3_bucket" "artifacts" {
  bucket = var.bucket_name

  tags = {
    Name        = "application-artifacts"
    Environment = "development"
    ManagedBy   = "Terraform"
  }
}

resource "aws_s3_bucket_public_access_block" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

output "artifact_bucket_name" {
  description = "Name of the created artifact bucket."
  value       = aws_s3_bucket.artifacts.bucket
}
```

The AWS provider obtains credentials through the standard AWS credential chain, such as environment variables, shared configuration, or an assigned workload role. Avoid hardcoding access keys in Terraform files.

3.3 Initialize, validate, plan, and apply

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
```

`terraform init` downloads provider plugins and initializes the working directory. `terraform fmt` formats configuration. `terraform validate` checks syntax and internal consistency. `terraform plan` previews changes. Applying a saved plan helps ensure the reviewed plan is the one executed.

After applying:

```
terraform output
terraform state list
```

Terraform state maps configuration resources to real infrastructure objects. Treat state as sensitive because it can contain secrets and infrastructure metadata.

3.4 Remote state and state locking

A team should avoid relying on separate local state files. Remote state allows authorized collaborators and automation to use a shared state location. A backend may also provide locking to prevent concurrent state modifications.

An illustrative S3 backend configuration is:

```
terraform {
  backend "s3" {
    bucket       = "company-terraform-state"
    key          = "platform/dev/terraform.tfstate"
    region       = "eu-north-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

The backend bucket must exist before Terraform initializes this backend. Restrict access to the state bucket, enable versioning, and configure appropriate encryption and retention. Backend options and locking support depend on the Terraform version and backend implementation.

3.5 Variables and environment separation

Use variables for values that differ by environment. Keep reusable infrastructure in modules and supply environment-specific values through separate variable files.

```
# variables.tf
variable "environment" {
  description = "Deployment environment."
  type        = string

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

# dev.tfvars
environment = "dev"
aws_region  = "eu-north-1"
```

Run a plan for the selected environment:

```
terraform plan -var-file="dev.tfvars"
```

Do not use workspaces as a substitute for a sound environment and access-control strategy when environments require materially different permissions or state boundaries.

3.6 Safe Terraform practices

- Commit `.tf` files and lock files to version control.
- Do not commit state files, plan files, or secret variable files.
- Review plans before applying changes.
- Use remote state with access controls and locking.
- Pin provider versions and review upgrades.
- Use modules for reusable patterns.
- Apply least privilege to the identity running Terraform.
- Run formatting, validation, and policy checks in CI.
- Protect production applies with approval gates.
- Back up state and understand recovery procedures.

---

4. Amazon Web Services Fundamentals

4.1 AWS regions and availability zones

AWS provides cloud services in geographic Regions. Many Regions contain multiple Availability Zones (AZs), which are physically separate locations engineered for fault isolation.

A resilient architecture can distribute application components across multiple AZs. Region selection should consider latency, service availability, regulatory requirements, data residency, and cost.

4.2 Identity and Access Management

AWS Identity and Access Management (IAM) controls access to AWS resources. An IAM policy defines allowed or denied actions on resources, optionally under conditions.

A narrowly scoped policy example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOneArtifactBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::company-artifacts",
        "arn:aws:s3:::company-artifacts/*"
      ]
    }
  ]
}
```

The policy grants read access to a specific bucket and its objects. Real policies should be tested and reviewed. Avoid broad permissions such as `Action: "*"` and `Resource: "*"` unless there is a justified administrative requirement.

Prefer temporary credentials and IAM roles over long-lived access keys. For human access, IAM Identity Center can provide centralized access to AWS accounts and roles.

4.3 Virtual Private Cloud

Amazon Virtual Private Cloud (VPC) provides a logically isolated network. A VPC has an IPv4 CIDR range, such as `10.0.0.0/16`, from which subnet ranges are allocated.

Typical network components include:

- **Public subnet:** Has a route to an Internet Gateway; resources still need suitable public addressing and security rules to be reachable.
- **Private subnet:** Does not have a direct route to an Internet Gateway.
- **Internet Gateway:** Enables internet connectivity for resources with suitable routes and addressing.
- **NAT Gateway:** Allows resources in private subnets to initiate outbound IPv4 connections without accepting unsolicited inbound internet connections.
- **Route table:** Determines where subnet traffic is directed.
- **Security group:** Stateful virtual firewall associated with network interfaces.
- **Network ACL:** Stateless subnet-level traffic filter.

Example route table entries:

Destination	Target	Purpose
`10.0.0.0/16`	`local`	VPC-internal traffic
`0.0.0.0/0`	Internet Gateway	Internet route for a public subnet

A secure design places application servers in private subnets where possible and exposes only required entry points, such as a load balancer, to the internet.

4.4 Compute and storage services

Amazon EC2 provides resizable virtual servers. Instance selection should consider CPU, memory, network throughput, storage, and workload characteristics.

Amazon S3 provides object storage. Common uses include static assets, backups, logs, and build artifacts. Use block public access, encryption, lifecycle rules, and bucket policies appropriate to the data.

Amazon EBS provides block storage volumes for EC2 instances. EBS volumes are typically attached to instances in the same Availability Zone.

Elastic Load Balancing distributes incoming traffic across registered targets. Application Load Balancers support HTTP and HTTPS routing, while Network Load Balancers operate at the transport layer.

4.5 Launching and inspecting an EC2 instance with the AWS CLI

The following commands illustrate how to inspect AWS identity and available resources. The account must already have appropriate credentials and permissions.

```
# Confirm which identity the CLI is using.
aws sts get-caller-identity

# List available EC2 instances in the configured region.
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].[InstanceId,State.Name,InstanceType]' \
  --output table

# List S3 buckets visible to the current identity.
aws s3 ls
```

Before creating resources, confirm the target account and Region. Use tags to identify ownership, environment, and cost center.

4.6 Monitoring, logging, and cost management

Amazon CloudWatch provides metrics, logs, alarms, and dashboards. CloudTrail records account activity and API events. AWS Config can track resource configuration and compliance.

Operational practices include:

- Create alarms for service health and resource saturation.
- Retain logs according to operational and compliance needs.
- Enable CloudTrail in the accounts and Regions that require audit coverage.
- Use cost allocation tags and budgets.
- Review idle resources and unattached storage.
- Prefer managed services when their operational benefits justify their cost.
- Test backup restoration, not only backup creation.

---

5. Continuous Integration and Continuous Delivery

5.1 CI/CD concepts

Continuous Integration (CI) is the practice of frequently integrating code changes and automatically validating them. Continuous Delivery (CD) keeps software in a deployable state and may require approval before production release. Continuous Deployment automatically releases qualifying changes to production.

A typical pipeline includes:

1. Source checkout.
2. Dependency installation.
3. Formatting and linting.

4. Unit and integration tests.
5. Security and dependency scanning.
6. Build and package.
7. Publish an immutable artifact or container image.
8. Deploy to a test environment.
9. Verify deployment health.
10. Promote to production with suitable controls.

A reliable pipeline should be repeatable, observable, secure, and fast enough to provide useful feedback.

5.2 GitHub Actions workflow

The following workflow runs checks for a Node.js application when changes are pushed or a pull request targets the `main` branch.

```
name: Node.js CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out source code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Run lint checks
        run: npm run lint --if-present

      - name: Run tests
        run: npm test --if-present

      - name: Build application
        run: npm run build --if-present
```

The workflow uses a read-only repository permission because it only checks code. A workflow that publishes packages or deploys infrastructure needs additional, narrowly scoped permissions.

For higher supply-chain assurance, pin third-party actions to reviewed full commit SHAs and update them through a controlled process.

5.3 Build and publish a container image

A container publishing job should use immutable tags, authenticate through a short-lived identity where possible, and avoid exposing credentials in logs.

Illustrative GitHub Actions steps:

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Log in to container registry
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Build and push image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ghcr.io/${{ github.repository }}:${{ github.sha }}
      ghcr.io/${{ github.repository }}:latest
```

The workflow needs suitable package permissions and repository settings. In production, consider publishing a digest, signing the image, generating a software bill of materials (SBOM), and scanning the image before deployment.

5.4 Deployment with environment protection

A production deployment should use an environment with required reviewers and scoped secrets. The deployment job should run only after validation and artifact publication have succeeded.

```
deploy-production:
  needs: test
  runs-on: ubuntu-latest
  environment: production
  permissions:
    contents: read
    packages: read

  steps:
    - name: Deploy application
      run: |
        echo "Deploy the tested image digest to production here."
```

The `echo` line is a placeholder, not a real deployment command. A real deployment step should authenticate securely to the target platform and deploy the exact image digest produced by the build job.

5.5 Pipeline security

- Use least-privilege `GITHUB_TOKEN` permissions.
- Store credentials in approved secrets or use OIDC federation.
- Do not print secrets or write them into build artifacts.
- Pin and review third-party actions.
- Scan dependencies and container images.
- Protect production environments with approvals.
- Separate build and deployment permissions.
- Avoid running untrusted pull-request code with privileged secrets.
- Keep deployment credentials short-lived where possible.

5.6 Reliability and observability

A pipeline is part of the production system. Teams should monitor job duration, failure rates, queue time, deployment frequency, and recovery time.

Useful practices include:

- Make build steps deterministic.
- Cache dependencies safely to reduce repeated work.
- Keep test output and deployment logs accessible.
- Use health checks after deployment.
- Support rollback to a known-good artifact.
- Avoid deploying mutable tags as the only release reference.
- Record the source commit and artifact digest for every release.

---

6. Cross-Topic Operational Scenarios

6.1 Deploying a containerized application

A common delivery path combines all five topics:

1. A developer commits application code.
2. CI installs dependencies and runs tests.
3. The pipeline builds a Docker image.
4. The image is scanned and published to a registry.
5. Terraform provisions or updates the required cloud infrastructure.
6. Kubernetes deploys the image and maintains the desired number of replicas.
7. AWS networking, IAM, storage, and monitoring services support the workload.
8. The pipeline verifies health and records the release.

Each stage has a distinct responsibility. Docker packages the application; Kubernetes orchestrates containers; Terraform manages infrastructure; AWS provides cloud resources; CI/CD automates validation and delivery.

6.2 Example incident: application is unreachable

A structured troubleshooting sequence avoids changing several layers at once.

```
# 1. Check whether the Kubernetes Deployment is healthy.
kubectl get deployment web-api
kubectl get pods -l app=web-api

# 2. Inspect recent application logs.
kubectl logs deployment/web-api --tail=100

# 3. Check the Service and its endpoints.
kubectl get service web-api
kubectl get endpoints web-api

# 4. Inspect recent cluster events.
kubectl get events --sort-by=.metadata.creationTimestamp

# 5. Confirm the cloud load balancer and network configuration
# using the relevant AWS console or AWS CLI commands.
```

Possible causes include a failing application process, incorrect container port, failed readiness probe, mismatched Service selector, missing load-balancer target registration, restrictive security-group rules, or a DNS problem.

6.3 Change management

Infrastructure and application changes should be traceable. Use version control, pull-request review, automated checks, documented approvals, and rollback plans. For infrastructure, review the Terraform plan before applying. For applications, retain the previous known-good image digest and deployment configuration.

---

7. Glossary

Term	Meaning
API	Application Programming Interface
AZ	Availability Zone
CI	Continuous Integration
CD	Continuous Delivery or Continuous Deployment, depending on context
CIDR	Classless Inter-Domain Routing notation for IP ranges
Container	An isolated process created from an image
IaC	Infrastructure as Code
IAM	Identity and Access Management
Image	A packaged filesystem and metadata used to create containers
Ingress	Kubernetes API resource for HTTP(S) routing, usually implemented by a controller
Pod	Smallest deployable Kubernetes workload unit
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
Registry	Service that stores and distributes container images
ReplicaSet	Kubernetes controller that maintains a set of Pod replicas
State file	Terraform record mapping configuration resources to managed objects
VPC	Virtual Private Cloud

---

8. Corpus Metadata and Retrieval Notes

This document is organized into five primary technical topics and two cross-topic reference sections.

Section	Topic	Main concepts
1	Docker	Images, containers, Dockerfiles, Compose, security
2	Kubernetes	Architecture, Deployments, Services, configuration, troubleshooting
3	Terraform	Providers, resources, state, plans, modules, remote state
4	AWS	IAM, VPC, EC2, S3, monitoring, cost management
5	CI/CD	GitHub Actions, testing, image publishing, deployment, security
6	Cross-topic scenarios	End-to-end delivery, troubleshooting, change management
7	Glossary	Shared terminology

For the framework comparison, use the same source document, embedding model, chunking policy, retrieval depth, and evaluation questions for each implementation. Record the extracted text and chunk metadata so results can be traced to the source section.

Recommended metadata for each chunk:

```
{
  "source": "devops-rag-corpus.md",
  "section": "2. Kubernetes Fundamentals",
  "topic": "Kubernetes",
  "content_type": "explanation",
  "language": "en"
}
```

Code blocks should remain intact where possible. If a chunking strategy splits code examples, record that behavior and compare it separately from a controlled benchmark that uses a shared set of preprocessed chunks.

End of corpus.